

Exp no:2

Developing and Deploying a Simple Chaincode

Aim

To develop, package, install, approve, commit, and execute a simple asset transfer chaincode in a Hyperledger Fabric test network using the JavaScript programming language

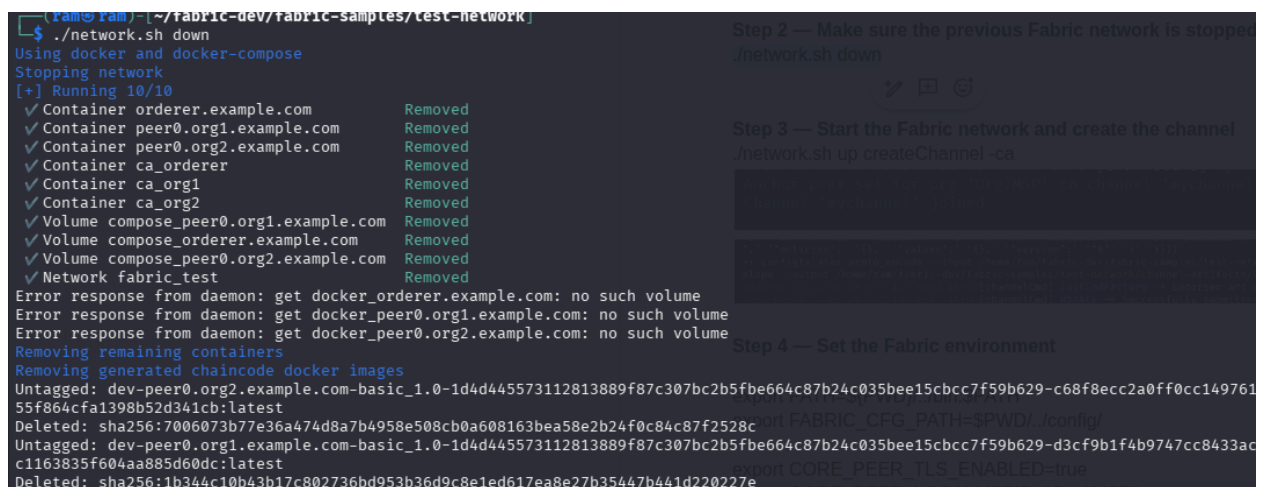
Procedure:

Step 1 — Go to the test network

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2 — Make sure the previous Fabric network is stopped

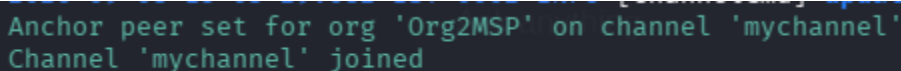
```
./network.sh down
```



```
(ram@ram) ~/fabric-dev/fabric-samples/test-network
$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] Running 10/10
  ✓ Container orderer.example.com      Removed
  ✓ Container peer0.org1.example.com   Removed
  ✓ Container peer0.org2.example.com   Removed
  ✓ Container ca_orderer               Removed
  ✓ Container ca_org1                  Removed
  ✓ Container ca_org2                  Removed
  ✓ Volume compose_peer0.org1.example.com Removed
  ✓ Volume compose_orderer.example.com Removed
  ✓ Volume compose_peer0.org2.example.com Removed
  ✓ Network fabric_test                Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f5f664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc14976155f864cfa1398b52d341cb:latest
Deleted: sha256:7006073b77e36a474d8a7b4958e508cb0a608163bea58e2b24f0c84c87f2528c
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f5f664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433acc1163835f604aa885d60dc:latest
Deleted: sha256:1b344c10b43b17c802736bd953b36d9c8e1ed617ea8e27b35447b441d220227e
```

Step 3 — Start the Fabric network and create the channel

```
./network.sh up createChannel -ca
```



```
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
```

```
"", "policies": {}, "values": {}, "version": "0" } } } } }
+ configtxlator proto_encode --input /home/ram/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update.pb --output /home/ram/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-09-08 15:03:29.674 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 15:03:29.682 IST 0002 INFO [channelCmd] update → Successfully submitted channel update
```

Step 4 — Set the Fabric environment

```
export PATH=${PWD}/../bin:$PATH
```

```
export FABRIC_CFG_PATH=$PWD/../config/
```

```
export CORE_PEER_TLS_ENABLED=true
```

```
export CORE_PEER_LOCALMSPID="Org1MSP"
```

```
export
```

```
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
```

```
export
```

```
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
```

```
export CORE_PEER_ADDRESS=localhost:7051
```

```
export
```

```
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

```
echo $ORDERER_CA
```

It should print the path to the orderer's TLS CA certificate.

```
(ram@ram) ~/fabric-dev/fabric-samples/test-network
$ echo $ORDERER_CA
/home/ram/fabric-dev/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

Step 5 — Deploy the JavaScript chaincode

```
./network.sh deployCC \
```

```
-ccl javascript \
```

```
-ccn basic \
```

```
-ccp ../asset-transfer-basic/chaincode-javascript
```

Step 6 — Verify that the chaincode is installed

peer lifecycle chaincode queryinstalled

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode queryinstalled
Installed chaincodes on peer:
Package ID: basic_1.0:1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbcc7f59b629, Label: basic_1.0
```

Step 7 — Initialize the ledger

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls \

--cafile "\$ORDERER_CA" \

-C mychannel \

-n basic \

-c '{"function": "InitLedger", "Args": []}'

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function": "InitLedger", "Args": []}'
2026-09-08 15:12:05.394 IST 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200
```

Step 8 — Query all assets

Now let's prove that the data is actually in the ledger.

Run:

peer chaincode query

Step 8 — Query all assets

peer chaincode query \

-C mychannel \

-n basic \

-c '{"Args": ["GetAllAssets"]}'

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":800,"Color":"Black","ID":"asset10","Owner":"David","Size":15}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Step 9 - Create asset10

peer chaincode invoke \

-o localhost:7050 \

--ordererTLSHostnameOverride orderer.example.com \

--tls \

```
--cafile "$ORDERER_CA" \  
-C mychannel \  
-n basic \  
-c '{"function":"CreateAsset","Args":["asset10","Black","15","Arun","800"]}' \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.c  
om-cert.pem" \  
--waitForEvent
```

Read it

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'
```

Transfer it to David

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile "$ORDERER_CA" \  
-C mychannel \  
-n basic \  
-c '{"function":"TransferAsset","Args":["asset10","David"]}' \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.c  
om-cert.pem" \  
--waitForEvent
```

Step - 10 verify

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'
```

```
(ram@ram)-[~/fabric-dev/fabric-samples/test-network]  
$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'  
{ "AppraisedValue":800,"Color":"Black","ID":"asset10","Owner":"David","Size":15}
```

against Org1.
For transactions/i